GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 18 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Greedy

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Greedy | 1 questions | tap a title to jump to it

M 1. Min boards to cover roof holes **PracHub 2026-02-12**

STUDY PATH: question 83 of 167 (Core 90)

Priority score 1.78 | Greedy | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 1 of 1

Min boards to cover roof holes

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 150/228

ABOUT THIS QUESTION

Topic: Core 90 - Greedy

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-05)

Original: https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

APPROACH

Greedy interval cover: sort hole positions, place a board starting at the first uncovered hole, skip all holes within its length, repeat.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask: are holes points on a line or cells in a grid, what is the board length, can boards be placed in either orientation, and must each board be axis aligned. The 1D case is a classic interval cover.

BRUTE FORCE TO SAY OUT LOUD

Try every placement of boards. Exponential.

OPTIMAL IDEA

1D with boards of length L: sort the holes, place a board starting at the first uncovered hole so it covers [h, h + L - 1], skip every hole inside it, repeat. The count is the answer, and the exchange argument shows starting at the leftmost uncovered hole is optimal. O(n log n). For a grid with horizontal boards only, do this per row. If boards can be placed in either orientation covering rows or columns, it becomes a minimum vertex cover in a bipartite graph (Konig's theorem), solved with matching.

DRY-RUN THESE CASES

No holes (0), one hole, holes spread more than L apart, adjacent holes, holes at the same position.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Boards of different lengths, or a limit on the number of boards.

PITFALLS

Starting a board in the middle of an uncovered gap. Not sorting the holes first.

HOW TO THINK ABOUT IT

Cover the leftmost hole first: any board that covers it can be slid right until it starts exactly at that hole without uncovering anything to its right, so start a board at the leftmost uncovered hole and skip everything it covers. Repeating that is optimal because starting later never covers more.

Basis: Written; interval covering (compare LeetCode 452 and 1326). Assumed rule (confirm it): holes are integer positions on a line, a board of length L covers positions p .. p + L - 1, boards may be placed anywhere; minimise the number of boards.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (len(holes) + 1)
best[0] = 0
for i in range(1, len(holes) + 1):
    for j in range(i):
        span = holes[i - 1] - holes[j] + 1
        if span <= L:  # one board covers j..i-1
            best[i] = min(best[i], best[j] + 1)
answer = best[len(holes)]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION · Greedy: start a board at the leftmost uncovered hole

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

FOLLOW-UP 2 Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

FOLLOW-UP 3 Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Min boards to cover roof holes
#
# This question evaluates competency in combinatorial
# optimization and problem modeling for coverage
# constraints, testing algorithmic reasoning, complexity
# analysis, and efficient representation of grid-based
# data.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You have only K boards: what is the smallest board
# length that covers all holes?
# 2. Boards of different lengths with a cost each.
# 3. Holes are in a grid and boards are 1 x L
# horizontally.
#
# INTERVIEW NOTES
# Cover the leftmost hole first: any board that covers it
# can be slid right until it starts exactly at that hole
# without uncovering anything to its right, so start a
# board at the leftmost uncovered hole and skip everything
# it covers. Repeating that is optimal because starting
# later never covers more.
#
# BRUTE FORCE (say it first): DP over the number of holes
# covered
# best[i] is the fewest boards that cover the first i
# holes: try every start j of the last board (it must
# cover holes j..i-1 within length L). Exact and the
# oracle; quadratic. Time O(n^2). Space O(n).


from bisect import bisect_left


# BRUTE FORCE (say it first): DP over the number of holes
# covered | time O(n^2)
def boards_dp(holes, L):
    h = sorted(set(holes))
    best = [0] + [float("inf")] * len(h)
    for i in range(1, len(h) + 1):
        for j in range(i):
            if h[i - 1] - h[j] < L:
                best[i] = min(best[i], best[j] + 1)
    return best[-1]
```

```
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count


# FOLLOW-UP 1: You have only K boards: what is the
# smallest board length that covers all holes?
# Answer: Binary search the length L and test with the
# greedy (is the number of boards at most K?); feasibility
# is monotone in L. O(n log range).
def smallest_length(holes, k):
    # only k boards: the shortest board length that still
    # covers everything (more length never hurts: binary
    # search)
    pts = sorted(set(holes))
    if not pts:
        return 0
    lo, hi = 1, pts[-1] - pts[0] + 1
    while lo < hi:
        mid = (lo + hi) // 2
        if min_boards(pts, mid) <= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 2: Boards of different lengths with a cost
# each.
# Answer: The greedy exchange argument breaks. Use DP over
# the sorted holes: best[i] = min over board types of
# best[j] + cost, where j is the first hole not covered by
```

```
# that board starting at hole i.
def min_cost(holes, boards):
    # boards = [(length, cost)], any number of each: a DP
    # over
    # the sorted holes (the greedy no longer works)
    pts = sorted(set(holes))
    INF = float("inf")
    best = [0] * (len(pts) + 1)
    for i in range(len(pts) - 1, -1, -1):
        best[i] = INF
        for length, cost in boards:
            # the first hole this board does not reach
            j = bisect_left(pts, pts[i] + length)
            best[i] = min(best[i], cost + best[j])
    return best[0]


# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L
# horizontally.
# Answer: Apply the greedy independently on every row. If
# boards may also be vertical it becomes a covering
# problem on a bipartite graph (König's theorem).
def boards_per_row(grid, length):
    # holes in a grid ('#'), horizontal boards: greedy per
    # row
    total = 0
    for row in grid:
        holes = [c for c in range(len(row))
                 if row[c] == "#"]
        total += min_boards(holes, length)
```

```
    return total


if __name__ == "__main__":
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1
    # with 2 boards the shortest length is 5 (covers 1..5,
    # 9..13)
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0
    # a long board costs 5, two short ones cost 3 each
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5
```

```
    roof = ["#.#.", "..##", "...."]
    assert boards_per_row(roof, 2) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
`[[1,0,1],[0,1,1]]`	`2`
`[[1,1],[1,1]]`	`0`